B.Tech (Computer Science and Engineering) – Semester VII

Subject: MACHINE LEARNING

Marks: 64

Assignment #2

SECTION – A (2 × 5 = 10 Marks)

Q1. [CO3] Define overfitting and underfitting in machine learning. Give one example of each.

Answer:

- **Overfitting:** When a model learns the noise and details of training data too well, leading to poor performance on test data.
  Example: A decision tree that grows too deep and perfectly classifies training data but fails on new data.
- **Underfitting:** When a model is too simple and fails to learn the underlying pattern in data.
  Example: Fitting a straight line to non-linear data like a sine curve.

Q2. [CO3] What is the role of a cost function in linear regression, and how does it measure model performance?

Answer:
The **cost function** (also called loss function) measures how far the predicted outputs are from the actual outputs.
In linear regression, the most common cost function is **Mean Squared Error (MSE):**

$$J(\theta) = \frac{1}{2m}\sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2$$

Lower value of cost function = better model performance.

Q3. [CO3] Differentiate between training error and test error.

Answer:

Training Error	Test Error
Error measured on data used to train the model	Error measured on unseen/test data
Usually lower	Usually higher
Does not indicate ability to generalize	Used to check overfitting/underfitting

---

Q4. [CO3] What is a hyperparameter? Give one example in the context of neural networks.

Answer:
A **hyperparameter** is a parameter set before the learning process, not learned from data.
Example: Learning rate (η), number of hidden layers, number of neurons, batch size, etc.

Q5. [CO3] Mention two key assumptions made in linear regression models.

Answer:

1. **Linearity:** Relationship between independent and dependent variables is linear.
2. **Homoscedasticity:** The variance of errors is constant across all values of inputs.

SECTION – B (3 × 7 = 21 Marks)

Q1. [CO3] Explain the bias-variance trade-off in machine learning. How does it affect a model's generalization performance? Illustrate with a diagram.

Answer:
The **bias-variance trade-off** explains the balance between two types of errors:

Bias	Variance
Error due to oversimplification	Error due to sensitivity to training data
High bias → Underfitting	High variance → Overfitting

✅ **Generalization is best when both are balanced.**

Graph Description:

- X-axis: model complexity
- Y-axis: error
- Training error decreases as complexity increases
- Test error decreases then increases (U-shape)

Q2. [CO3] Compare and contrast Principal Component Analysis (PCA) and Feature Selection. Discuss when each technique is preferable.

Answer:

PCA	Feature Selection
Creates new transformed features	Retains original features
Dimensionality reduction by projection	Dimensionality reduction by elimination
Unsupervised	Can be supervised or unsupervised
Useful when features are correlated	Useful when we need interpretability

✅ **Use PCA** when data has high dimensionality and correlation (e.g., image compression).
✅ **Use Feature Selection** when original features must be preserved (e.g., medical data attributes).

Q3. [CO3] Describe the steps involved in evaluating a machine learning model using cross-validation. Why is it important?

Answer:

✅ **Steps of k-fold Cross Validation**

1. Shuffle dataset
2. Split into *k* equal folds
3. Train model on *(k–1)* folds
4. Test on remaining fold
5. Repeat *k* times and take average score

✅ **Why important?**

✓ Gives more reliable performance estimation
✓ Reduces bias due to single train-test split
✓ Useful for small datasets

Example: 5-fold cross validation on loan prediction dataset → Accuracy averaged over 5 runs.

SECTION – C (3 × 11 = 33 Marks)

Q1. [CO4] Discuss the architecture and learning process of a multilayer neural network. Explain the role of backpropagation and gradient descent with suitable equations and diagrams.

Answer:

A **Multilayer Neural Network (MLP)** is an artificial neural network consisting of an **input layer, one or more hidden layers, and an output layer**. Each node in a layer is connected to nodes in the next layer through weights.

✅ **Architecture of MLP**

Input Layer → Hidden Layer(s) → Output Layer

Each neuron performs:

$$z = W \cdot X + b$$
$$a = f(z)$$

Where:

- W= weights
- b= bias
- $f(.)$= activation function (ReLU, Sigmoid, Tanh, etc.)

✅ **Forward Propagation**

The input flows forward through network:

$$a^{(l)} = f(W^{(l)}a^{(l-1)} + b^{(l)})$$

✅ **Cost Function**

For classification (using cross entropy):

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}[y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))]$$

✅ **Backpropagation (Error Propagation)**

Used to update weights by calculating gradient of cost function w.r.t. weights.

$$\delta^{(l)} = (W^{(l+1)})^T\delta^{(l+1)} \odot f'(z^{(l)})$$

Final weight update:

$$W := W - \alpha \frac{\partial J}{\partial W}$$

Where **α = learning rate**

✅ **Gradient Descent Update Rule**

$$W_{new} = W_{old} - \alpha \cdot \frac{\partial J}{\partial W}$$

✅ **Learning Process Summary**

Step	Description
1	Initialize weights & biases randomly
2	Forward propagate input → get prediction
3	Compute loss using cost function
4	Backpropagate error to find gradients
5	Update weights using gradient descent
6	Repeat for multiple epochs until convergence

✅ **Advantages**

✓ Can learn non-linear patterns
✓ Works for classification & regression
✓ Support for deep learning (CNN, RNN)

✅ **Limitations**

❌ Computationally expensive
❌ Requires large data for training
❌ Prone to overfitting without regularization

Q2. [CO4] Explain Regularized Logistic Regression. Discuss the motivation behind regularization, how it helps prevent overfitting, and derive the cost function used for both L1 and L2 regularization.

Answer:

Regularization is a technique used to **reduce overfitting** by adding a penalty term to the cost function that **discourages large weights** in the model.

✅ Motivation

- Without regularization, logistic regression may **fit noise** in the data → overfitting.
- Regularization forces the model to be **simpler and more generalizable**.

✅ Original Logistic Regression Cost Function

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m} [y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))]$$

✅ L2 Regularization (Ridge Regression)

Penalty term = sum of squared weights:

$$J(\theta)_{L2} = J(\theta) + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

- Shrinks weights smoothly
- Does **not** set weights to zero
- Good for multicollinearity

✅ L1 Regularization (Lasso Regression)

Penalty = sum of absolute weights:

$$J(\theta)_{L1} = J(\theta) + \frac{\lambda}{m}\sum_{j=1}^{n} |\theta_j|$$

- Can set some weights exactly **zero** → performs **feature selection**

✅ Effect of Regularization Parameter (λ)

λ value	**Effect**
λ = 0	No regularization → may overfit

λ value	Effect
λ small	Light penalty
λ high	Strong penalty → underfitting possible

✅ **Key Differences**

Feature	L1	L2
Weight values	Some become zero	All shrink gradually
Feature selection	Yes	No
Optimization	Non-smooth	Smooth (differentiable)

Q3. [CO4] Describe the working of the Support Vector Machine (SVM) algorithm. Explain the concept of margin maximization and the role of kernel functions in handling non-linear data.

Answer:

SVM is a **supervised learning algorithm** used for classification and regression. It finds the **best separating hyperplane** that maximizes margin between classes.

✅ **Margin Maximization**

- Margin = distance between separating hyperplane and nearest data points (support vectors)
- Objective: maximize margin = better generalization

Optimal hyperplane formula:

$$w^T x + b = 0$$

Margin width:

$$\frac{2}{|| w ||}$$

SVM optimization:

$$\min \frac{1}{2} || w ||^2 \text{ subject to } y_i(w^T x_i + b) \geq 1$$

✅ **Soft Margin SVM**

Allows some misclassifications using penalty term **C**:

$$\min \frac{1}{2} || w ||^2 + C \sum_{i=1}^{m} \xi_i$$

✅ Kernel Trick (for Non-linear Data)

Instead of mapping data manually into higher dimensions, kernel function computes similarity in high-D space.

Common kernels:

Kernel	Formula	Use Case
Linear	$x \cdot x'$	Linearly separable data
Polynomial	$(x \cdot x' + 1)^d$	Medium complexity
RBF (Gaussian)	(e^{-\gamma	
Sigmoid	$\tanh(\alpha x^T x' + c)$	Neural network-like

✅ Advantages

✓ Works well in high-dimensional space
✓ Effective when number of features > samples
✓ Kernel trick handles non-linearity

✅ Limitations

❌ Slow for large datasets
❌ Choosing right kernel and parameters is difficult
❌ No probabilistic output (unless modified)